



CORPORACIÓN UNIVERSITARIA DEL HUILA – CORHUILA

Laboratorio de Conversión Analógico-Digital (ADC) en ESP32 con MicroPython

Integrantes:

Bedoya Montealegre Brayan Smith

Molina Fierro Jhon Sebastián

Facultad De Ingeniería, Corporación Universitaria del Huila - CORHUILA

Microcontroladores

Docente: Cesar Augusto Pérez Tafur

25 de septiembre del 2025

2 de octubre del 2025

Contenido

RESUMEN	3
INTRODUCCION	4
OBJETIVOS	4
Objetivo general.....	4
Objetivos Específicos	4
METODO.....	5
Participantes.....	5
Materiales	5
Procedimiento del laboratorio	5
CONCLUSIÓN	10
REFERENCIAS.....	11

Lista de figuras

Figura 1 – Potenciómetro.....	6
Figura 2 - Código del potenciómetro	6
Figura 3 - Lectura de temperatura	7
Figura 4 - Código de Lectura de temperatura.....	7
Figura 5 - Filtrado de ruido	8
Figura 6 - Código de filtrado de ruido.....	8
Figura 7 - Sistema de temperatura	9
Figura 8 - Sistema de temperatura	9
Figura 9 - Código de sistema de temperatura	9
Figura 10 - Mini Dara Logger.....	10
Figura 11 - Código de Mini Data Logger.....	10

RESUMEN

En esta tercera práctica de laboratorio se exploraron las capacidades del conversor analógico-digital (ADC) del microcontrolador ESP32-S3, un recurso fundamental en el desarrollo de sistemas embebidos donde se requiere la adquisición de señales analógicas provenientes de sensores y dispositivos externos. Mediante el uso del entorno de programación Thonny y el lenguaje MicroPython como se ha venido usando, así como su aplicación práctica en la lectura de un potenciómetro y un sensor de temperatura LM35.

Durante el desarrollo de este laboratorio, mi grupo trabajó de manera independiente y colaborativa, donde uno de los aspectos más destacados de la práctica fue la lectura en tiempo real de señales analógicas, evidenciada en la variación del voltaje captado por el potenciómetro y en la conversión de la salida del sensor LM35 en valores de temperatura. Adicionalmente, se implementaron técnicas de filtrado digital por promediado, aquí, los retos planteados exigieron creatividad y lógica, incluyendo el diseño para el control de un LED en función de la temperatura y proceso de datos en tiempo real.

Este laboratorio no solo sirvió como una introducción práctica al uso del ADC en el ESP32-S3, sino que también consolidó el aprendizaje de conceptos clave de señales analógicas, digitalización, filtrado y registro de datos. La integración de hardware y software permitió a mi grupo de trabajo experimentar con situaciones reales.

In this third lab, we explore the capabilities of the ESP32-S3 microcontroller's analog-to-digital converter (ADC), a fundamental resource in the development of embedded systems that require the acquisition of analog signals from sensors and external devices. This is achieved through the use of the Thonny programming environment and the MicroPython language, as previously used, and through its practical application in reading a potentiometer and an LM35 temperature sensor.

During this lab, my group worked independently and collaboratively. One of the highlights of the lab was the real-time reading of analog signals, evidenced by the variation in the voltage captured by the potentiometer and the conversion of the LM35 sensor output into temperature values. In addition, digital

filtering techniques were implemented. Here, the challenges posed required creativity and logic, including the design for controlling an LED based on temperature and processing data in real time. This lab not only served as a practical introduction to using the ADC on the ESP32-S3, but also consolidated the learning of key concepts related to analog signals, digitization, filtering, and data logging. The integration of hardware and software allowed my team to experiment with real-world situations.

INTRODUCCION

Dentro de este laboratorio, se examinó la operación del convertidor de señal analógica a digital (ADC) del ESP32-S3, investigando el modo en que transforma señales de tipo analógico en datos digitales que se pueden usar en la programación integrada.

Usando ejemplos reales, con un potenciómetro y un sensor de temperatura LM35, se asimilaron ideas elementales como el muestreo, la resolución y el filtrado digital, utilizados en la lectura, el procesamiento y el análisis de los datos al instante.

In this lab, the operation of the ESP32-S3 analog-to-digital converter (ADC) was examined, investigating how it transforms analog signals into digital data that can be used in embedded programming.

Using real-world examples, including a potentiometer and an LM35 temperature sensor, we learned basic ideas such as sampling, resolution, and digital filtering, which are used to read, process, and analyze data instantly.

OBJETIVOS

Objetivo general: Comprender y aplicar el funcionamiento del conversor analógico-digital (ADC) en el microcontrolador ESP32-S3, utilizando el lenguaje MicroPython para la lectura, procesamiento y análisis de señales analógicas.

Objetivos Específicos:

1. Implementar programas en MicroPython para la lectura de valores analógicos provenientes de un

potenciómetro y un sensor LM35.

2. Analizar los efectos del ruido en las mediciones.
3. Desarrollar un sistema de control simple basado en histéresis para activar o desactivar un LED según la temperatura medida.
4. Implementar un mini data logger que registre, almacene y procese datos en tiempo real, calculando valores mínimos, máximos y promedio.

METODO

Participantes: **Brayan Smith Bedoya Montealegre y Jhon Sebastián Molina Fierro** estudiantes del curso Microcontroladores, cursando el sexto semestre de Ingeniería de sistemas (Edad:18-19, M=18.5; SD=0.71). Se formaron grupos de 5 integrantes como máximo. Ambos trabajaron en un mismo grupo de forma colaborativa y contaban con conocimientos básicos en electrónica y programación en Python.

Materiales:

- Resistencias 220 Ohmios
- Leds de colores surtidos
- Pulsadores
- Jumpers
- Cable Tipo-C – Tipo-C
- Computador laptop
- Protoboard
- Sensor LM35

Procedimiento del laboratorio:

Ejemplo 1 – Lectura de potenciómetro: Circuito: Potenciómetro conectado a 3.3V y GND, salida al GPIO4.

The image shows a MicroPython IDE interface. At the top, the title bar reads "Thomys - C:\Programas\Ingeniería en Sistemas\Semestre 6\Microcontroladores\Semana 8\laboratorio 3.py @ 4:16". The menu bar includes "Fichero", "Editar", "Visualizar", "Ejecutar", "Herramientas", and "Ayuda". Below the menu is a toolbar with icons for file operations and execution. The main editor window displays a Python script named "laboratorio 3.py".

```

1 from machine import Pin, ADC
2 import time
3
4 adc = ADC(Pin(3))
5 adc.atten(ADC.ATTN_11DB)
6 adc.width(ADC.WIDTH_12BIT)
7
8 VREF = 3.3
9
10 while True:
11     raw = adc.read()
12     voltaje = (raw / 4095) * VREF
13     print("Raw=", raw, " Voltaje:", round(voltaje,3), "V")
14     time.sleep(0.5)

```

Below the editor is a console window with the command prompt "MicroPython (ESP32) >". The command ">>> \$@run -c \$EDITOR_CONTENT" has been executed, resulting in the following output:

```

MicroPython (ESP32) >
>>> $@run -c $EDITOR_CONTENT
Raw: 1020 Voltaje: 1.567 V
Raw: 1246 Voltaje: 1.516 V
Raw: 1448 Voltaje: 1.379 V
Raw: 1548 Voltaje: 1.37 V
Raw: 1849 Voltaje: 1.571 V
Raw: 1849 Voltaje: 1.571 V
Raw: 1552 Voltaje: 1.573 V
Raw: 1555 Voltaje: 1.573 V
Raw: 1501 Voltaje: 1.572 V
Raw: 1555 Voltaje: 1.575 V
Raw: 1551 Voltaje: 1.572 V
Raw: 1601 Voltaje: 1.572 V
Raw: 1849 Voltaje: 1.571 V
Raw: 1552 Voltaje: 1.573 V
Raw: 1601 Voltaje: 1.572 V
Raw: 6005 Voltaje: 2.3 V
Raw: 6095 Voltaje: 2.3 V
Raw: 6095 Voltaje: 3.3 V
Raw: 6095 Voltaje: 3.3 V
...

```

Ejemplo 2 – Lectura de sensor LM35 (temperatura):

```
Thonny - C:\Program Files\Microcontroladores\Semana 6\laboratorio 3.py @ 3:16
Fichero  Editor Visualizar Ejecutar Herramientas Ayuda

laboratorio 3.py
1 from machine import Pin, ADC
2 import time
3 adc = ADC(Pin(0))
4 adc.atten(ADC.ATTN_120DB)
5 adc.width(ADC.WIDTH_12BIT)
6 VREF = 3.3
7 def leer_temp():
8     raw = adc.read()
9     volt = (raw / 4095) * VREF
10    temp_c = volt / 0.01
11    return raw, volt, temp_c
12 while True:
13     raw, volt, temp = leer_temp()
14     print("Raw:", raw, " Voltage:", round(volt,3), "V", " Temp:", round(temp,3),
15           "°C")
16     time.sleep(1)
```

Console

```
Raw: 770 Voltage: 0.512 V Temp: 51.2 °C
Raw: 780 Voltage: 0.533 V Temp: 53.3 °C
Raw: 790 Voltage: 0.538 V Temp: 53.8 °C
Raw: 800 Voltage: 0.543 V Temp: 54.3 °C
Raw: 780 Voltage: 0.512 V Temp: 51.2 °C
Raw: 790 Voltage: 0.517 V Temp: 51.7 °C
Raw: 790 Voltage: 0.528 V Temp: 52.8 °C
Raw: 770 Voltage: 0.528 V Temp: 52.8 °C
Raw: 770 Voltage: 0.528 V Temp: 52.8 °C
Raw: 770 Voltage: 0.513 V Temp: 51.3 °C
Raw: 770 Voltage: 0.528 V Temp: 52.8 °C
Raw: 770 Voltage: 0.528 V Temp: 52.8 °C
Raw: 770 Voltage: 0.507 V Temp: 50.7 °C
Raw: 707 Voltage: 0.513 V Temp: 51.3 °C
Raw: 700 Voltage: 0.513 V Temp: 51.3 °C
Raw: 740 Voltage: 0.507 V Temp: 50.7 °C
Raw: 740 Voltage: 0.533 V Temp: 53.3 °C
Raw: 740 Voltage: 0.516 V Temp: 51.6 °C
Raw: 740 Voltage: 0.516 V Temp: 51.6 °C
Raw: 740 Voltage: 0.513 V Temp: 51.3 °C
Raw: 740 Voltage: 0.513 V Temp: 51.3 °C
```

Ejemplo 3 – Filtrado de ruido:

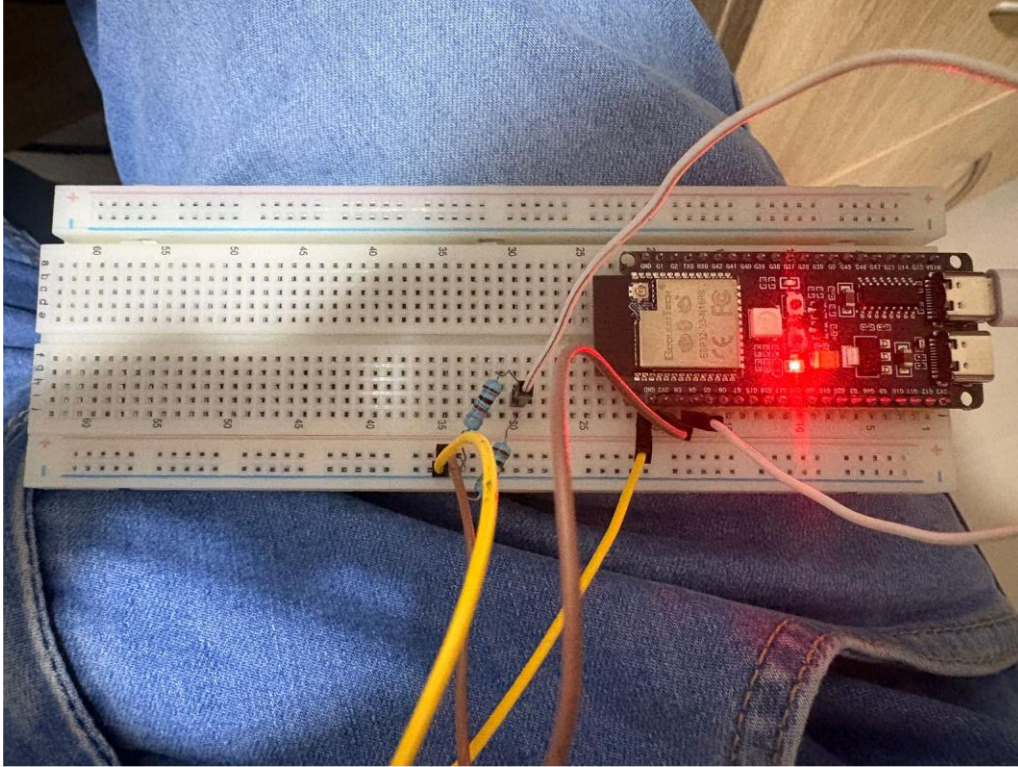


Figura 5 - Filtrado de ruido

Thorny - C:\Pregrado Ingeniería en Sistemas\Semestre 6\Microcontroladores\Semana 6\Laboratorio 3.py @ 3:1

Fichero Editar Visualizar Ejecutar Herramientas Ayuda

Laboratorio 3.py *

```

1 import machine
2 import time
3
4 adc = machine.ADC(machine.Pin(4))
5 adc.atten(machine.ADC.ATTN_110dB)
6 adc.width(machine.ADC.WIDTH_12BIT)
7
8 VREF = 3.3
9
10 def leer_filtrado(n=20):
11     suma = 0
12     for i in range(n):
13         suma += adc.read()
14     return suma // n
15
16 while True:
17     valor = leer_filtrado()
18     voltaje = (valor / 4095) * VREF
19     print("Filtrado:", valor, " Voltaje:", round(voltaje, 3), "V")
20     time.sleep(0.5)
21

```

Console

```

Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1907 Voltaje: 1.537 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1907 Voltaje: 1.537 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1907 Voltaje: 1.537 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1907 Voltaje: 1.537 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1906 Voltaje: 1.536 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1905 Voltaje: 1.535 V
Filtrado: 1904 Voltaje: 1.534 V

```

MicroPython (ESP32) - USB Serial @ COM3

Figura 6 - Código de filtrado de ruido

Reto inicial: Diseñar un sistema que lea temperatura con LM35, encienda un LED en GPIO2 si la temperatura supera 30°C y lo apague si baja de 28°C. Usar Timer con muestreo cada 200 ms.

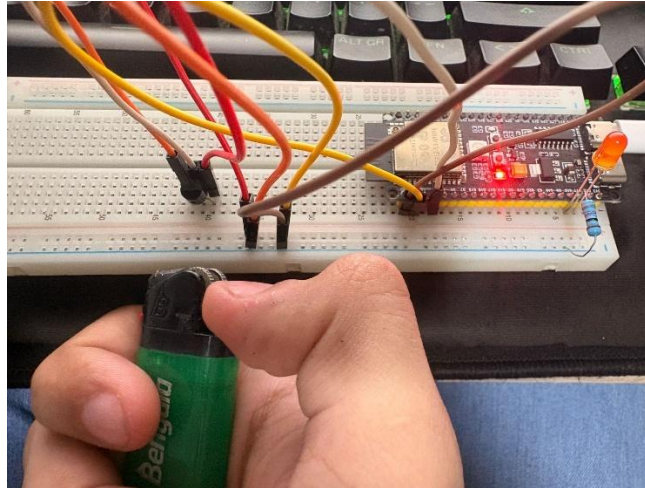


Figura 7 - Sistema de temperatura

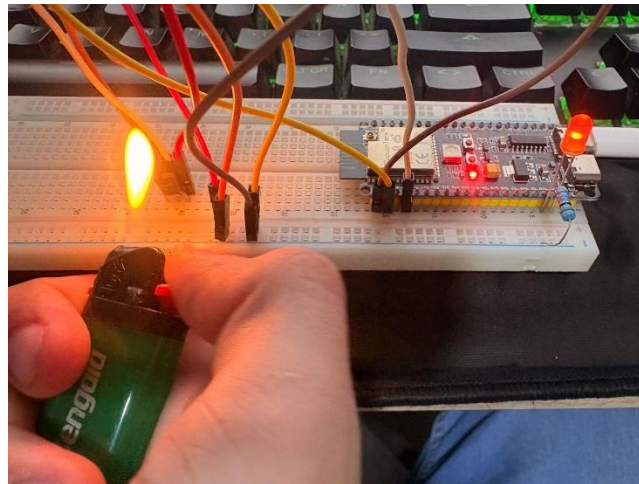


Figura 8 - Sistema de temperatura

```

Thonny - C:\Pegado Ingenieria en Sistemas\Semestre 6\Microcontroladores\Semana 6\Laboratorio 3.py 31:16
Fichero Editor Visualizar Ejecutar Herramientas Ayuda

Laboratorio 3.py
1 import machine
2 import time
3
4 adc = machine.ADC(machine.Pin(4))
5 adc.atten(machine.ADC.ATTN_11DB)
6 adc.width(machine.ADC.WIDTH_12BIT)
7
8 led = machine.Pin(11, machine.Pin.OUT)
9
10 TEMP_ON = 31.0
11 TEMP_OFF = 28.0
12
13 VREF = 3.3
14
15 def leer_temp():
16     valor = adc.read()
17     voltaje = (valor / 4095) * VREF
18     temp = voltaje * 100
19     return temp
20
21 estado_led = False
22
23 def muestreo(timer):
24     global estado_led
25     temp = leer_temp()
26     print("Temperatura:", round(temp, 2), "°C")
27
28     if temp >= TEMP_ON and not estado_led:
29         led.on()
30         estado_led = True
31         print("¡LED ENCENDIDO (Temp >= 30°C)")
32     elif temp <= TEMP_OFF and estado_led:
33         led.off()
34         estado_led = False
35         print("LED APAGADO (Temp <= 28°C)")
36
37 timer0 = machine.Timer(0)
38 timer0.init(period=200, mode=machine.Timer.PERIODIC, callback=muestreo)
39
40
41 Console
42 -temperatura: 27.8 °C
43 Temperatura: 27.8 °C
44 Temperatura: 27.89 °C
45 Temperatura: 27.86 °C
46 Temperatura: 27.89 °C
47 Temperatura: 27.86 °C
48 Temperatura: 27.84 °C
49 Temperatura: 27.89 °C
50 Temperatura: 27.8 °C
51
MicroPython (ESP32) - USB Serial @ COM5

```

Figura 9 - Código de sistema de temperatura

Reto avanzado – Mini Data Logger: Implementar un sistema que muestree un potenciómetro cada 100 ms, almacene los últimos 50 valores y calcule mínimo, máximo y promedio en tiempo real.

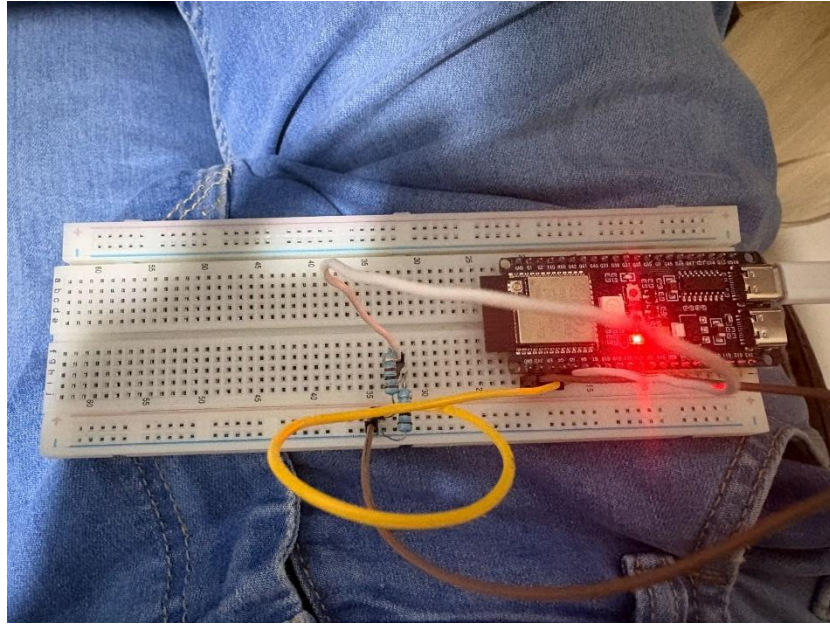


Figura 10 - Mini Data Logger

```

1 from machine import Pin, ADC, Timer
2 import time
3
4 adc = ADC(Pin(4))
5 adc.atten(ADC.ATTN_11DB)
6 adc.width(ADC.WIDTH_12BIT)
7
8 VREF = 3.3
9 data = []
10
11 def muestreo(timer):
12     global data
13     raw = adc.read()
14     data.append(raw)
15     if len(data) > 50:
16         data.pop(0)
17
18 t = Timer(0)
19 t.init(period=100, mode=Timer.PERIODIC, callback=muestreo)
20
21 try:
22     while True:
23         time.sleep(1)
24         if data:
25             minimo = min(data)
26             maximo = max(data)
27             promedio = sum(data) / len(data)
28             print("Min:", minimo,
29                 "Max:", maximo,
30                 "Promedio:", round(promedio, 2))
31 except KeyboardInterrupt:
32     t.deinit()
33     print("Detenido por el usuario")
34

```

Console

```

Min: 1921 Max: 1941 Promedio: 1939.24
Min: 1937 Max: 1943 Promedio: 1939.36
Min: 1935 Max: 1943 Promedio: 1939.44
Min: 1935 Max: 1943 Promedio: 1939.36
Min: 1935 Max: 1941 Promedio: 1939.38
Min: 1937 Max: 1941 Promedio: 1939.26
Min: 1937 Max: 1941 Promedio: 1939.34
Min: 1937 Max: 1941 Promedio: 1939.24
Min: 1935 Max: 1941 Promedio: 1939.34
Min: 1935 Max: 1941 Promedio: 1939.38

```

MicroPython (ESP32) • USB Serial @ COM3

Figura 11 - Código de Mini Data Logger

CONCLUSIÓN

El desarrollo de este laboratorio permitió comprender de manera práctica el uso del ADC en el ESP32-S3, consolidando los conceptos de muestreo, resolución y filtrado digital aplicados a señales analógicas.

A través de la lectura de un potenciómetro y un sensor de temperatura LM35, se evidenció cómo los datos analógicos pueden convertirse en información útil para la toma de decisiones en sistemas embebidos.

Además, la implementación de retos como el control de un LED por histéresis y el diseño de un mini data logger fortaleció las habilidades de programación y análisis, integrando hardware y software en un entorno real.

Esta experiencia no solo reafirmó los fundamentos teóricos, sino que también aportó una base sólida para futuros proyectos en el campo de la adquisición y procesamiento de señales en IoT y sistemas digitales.

REFERENCIAS

Random Nerd Tutorials. “ESP32/ESP8266 Analog Readings with MicroPython.” Recuperado de <https://randomnerdtutorials.com/esp32-esp8266-analog-readings-micropython/> — explicación del uso de ADC en ESP32 con MicroPython y lectura de valores analógicos. [Random Nerd Tutorials](https://randomnerdtutorials.com/esp32-esp8266-analog-readings-micropython/)

Luis Llamas. “How to Read Analog Inputs (ADC) in MicroPython.” Recuperado de <https://www.luisllamas.es/en/analog-entries-adc-micropython/> — ejemplos de configuración de resolución y atenuación en MicroPython. [Luis Llamas](https://www.luisllamas.es/en/analog-entries-adc-micropython/)

I Programmer. “ESP32 In MicroPython: Analog Input.” Recuperado de <https://www.i-programmer.info/programming/hardware/16686-esp32-in-micropython-analog-input.html> — detalles sobre precisión, calibración y limitaciones del ADC. [i-programmer.info](https://www.i-programmer.info/programming/hardware/16686-esp32-in-micropython-analog-input.html)

Microcontrollers Lab. “ESP32/ESP8266 ADC with MicroPython – Measure Analog Readings.” Recuperado de <https://microcontrollerslab.com/esp32-esp8266-adc-micropython-measure-analog-readings/> — guía práctica para medir señales analógicas con el ADC en MicroPython. [Microcontrollers Lab](https://microcontrollerslab.com/esp32-esp8266-adc-micropython-measure-analog-readings/)

ElectronicWings. “LM35 Sensor Interfacing with ESP32.” Recuperado de <https://www.electronicwings.com/esp32/lm35-sensor-interfacing-with-esp32> — uso del sensor LM35 con ESP32 y consideraciones prácticas. [electronicwings.com](https://www.electronicwings.com/esp32/lm35-sensor-interfacing-with-esp32)

Microcontrollers Lab. “LM35 Temperature Sensor with ESP32 Web Server.” Recuperado de <https://microcontrollerslab.com/lm35-temperature-sensor-with-esp32-web-server/> — relación entre resolución del ADC del ESP32 y la salida del LM35, limitaciones y soluciones. [Microcontrollers Lab](https://microcontrollerslab.com/lm35-temperature-sensor-with-esp32-web-server/)

Documentación oficial de MicroPython. “class ADC – analog to digital conversion.” Recuperado de <https://docs.micropython.org/en/latest/library/machine.ADC.html> — especificaciones del módulo ADC en MicroPython. [docs.micropython.org](https://docs.micropython.org/en/latest/library/machine.ADC.html)

UIFlow (MicroPython). “ADC (analog to digital conversion).” Recuperado de <https://uiflow-micropython.readthedocs.io/en/2.2.3/hardware/adc.html> — mapeo de pines ADC en ESP32 y ESP32-S3. [uiflow-micropython.readthedocs.io](https://uiflow-micropython.readthedocs.io/en/2.2.3/hardware/adc.html)